

開発ツール整備

開発ツール整備

Claude PR レビュー / lefthook / ruff の導入

Hyonun Kin

2026-04-29

目的・背景

phase 3 までで MVP の API は揃った。一方で、複数フェーズを跨いだ開発を続ける中で以下の課題が見えてきた：

- LLM が生成したコードを毎回手で読む工数が多い（前回採点で「LLM 生成コードを理解・指摘できる」が弱点）
- フォーマットや lint の不統一が PR レビューでの本筋議論を邪魔する
- テストの実行忘れに気付かないまま push してしまう

そこで今回は **ptw-respec の運用構成を参考に**、開発フローを支える3点を整備する：

1. **Claude PR レビュー**：PR を出した瞬間に Claude が自動レビュー → 自分が見落とした観点を自動で拾ってもらう
2. **lefthook**：commit 時に lint/format、push 時にテスト → 機械的に防げる事故を消す
3. **ruff**：lint と formatter を1つに統一 → 設定の重複を無くす

要件

機能要件：

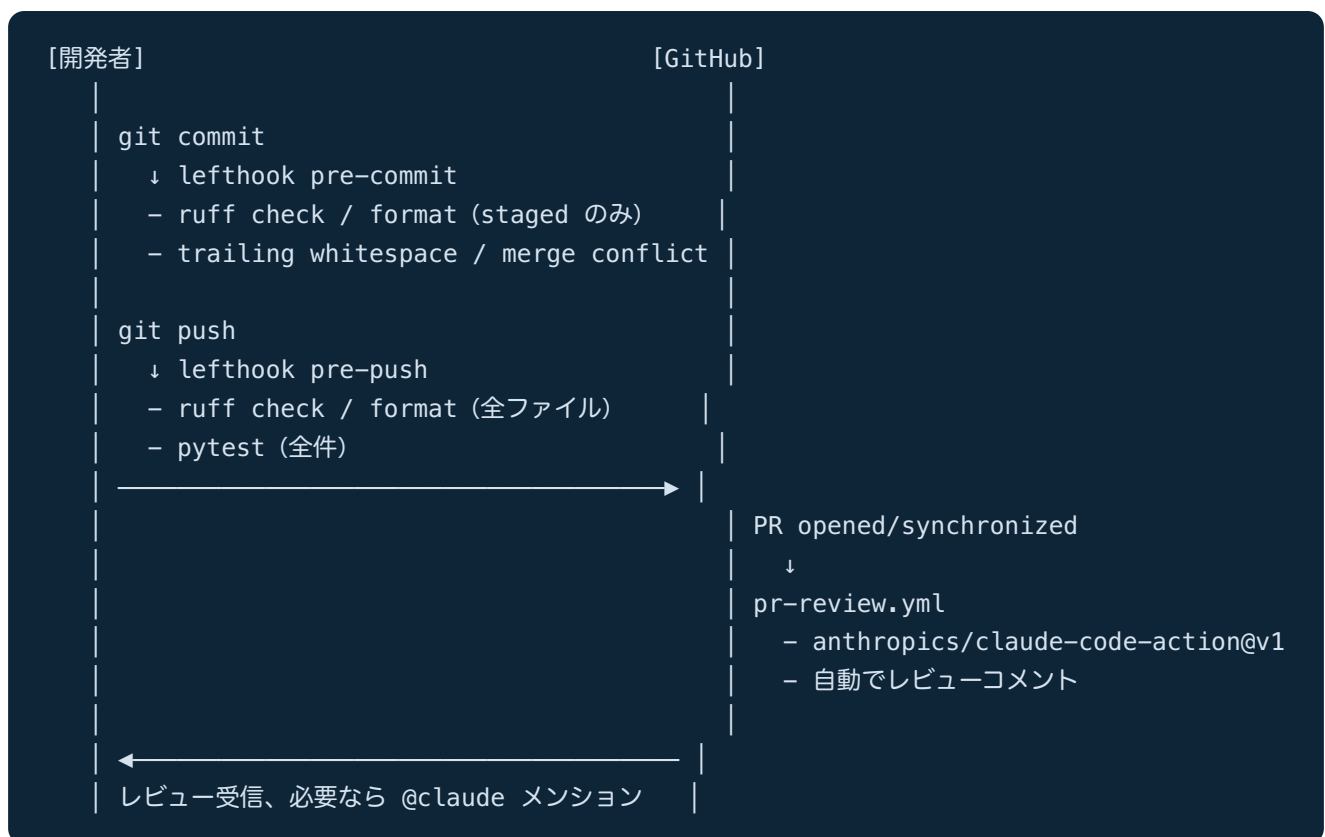
- `develop` / `main` 宛の PR が開かれると Claude が自動でレビューコメントを書く
- PR コメントで `@claude` をメンションすると追加レビューや質問対応が走る
- `git commit` 時に staged な Python ファイルに対して ruff の lint/format チェックが走る
- `git push` 時に backend の全テストと全ファイル lint/format が走る
- 開発者は `lefthook install` を1回打つだけで上記が有効になる

非機能要件：

- 既存コードを ruff の規則に通る状態にする（テスト 14 件は変わらず pass）
- lefthook は失敗時に commit/push をブロックする（事故を防ぐ）
- `LEFTHOOK=0` や `--no-verify` で一時的にスキップできる（緊急時のため）
- Claude PR レビューは Plan の枠で動かす（API 別契約しない）

設計

全体図



Claude PR レビュー

`.github/workflows/pr-review.yml` を新設。トリガーは：

- `pull_request` (opened / synchronize / ready_for_review / reopened) で `develop` / `main` 宛
- `issue_comment` で本文に `@claude` を含む場合 (PR コメント経由の追加レビュー)
- `pull_request_review_comment` 同上

レビュー観点のプロンプトで明示的に固定し、本リポの `CLAUDE.md` と `docs/architecture.md` を読ませる：

- 正確性・ロジック（`async / await`、エラー伝播、エッジケース）
- セキュリティ（SQLi / XSS / 認可バイパス / 機密漏洩）
- アーキテクチャ・規約（並び順、`docstring` 形式、ファイル分割の閾値）
- 設計判断との一貫性（永続化しない、JWT + Cookie、Gemini-2.5-flash-lite）
- テスト（mock 経由か、エッジケース網羅、認証失敗パス）

lefthook

`lefthook.yml` を repo ルートに配置。**commit と push で粒度を分ける**：

- **pre-commit（速い）**：staged ファイルだけに lint/format。3秒以内に終わる想定
- **pre-push（遅い）**：全テスト + 全ファイル lint。20秒程度

時間がかかる検査を pre-push に逃がすことで、こまめな commit を阻害しない設計にした。

ruff

backend の dev 依存に追加。1つのツールで lint と formatter を兼ねるのが選定理由（black + isort + flake8 の3点を1つに集約）。

設定（`pyproject.toml` 内）：

```
[tool.ruff]
line-length = 120          # 日本語 docstring を考慮して余裕を持たせる
target-version = "py313"

[tool.ruff.lint]
select = ["E", "F", "W", "I", "B", "UP"]
ignore = ["B008"]          # FastAPI の Depends() 引数デフォルトを許容

[tool.ruff.format]
quote-style = "double"
```

B008 を ignore する理由：FastAPI では `def login(session: AsyncSession = Depends(get_session))` のように引数デフォルトに関数呼び出しを書くのが正規パターン。ruff のデフォルトはこれを警告するが、フレームワーク側の規約に従う。

実装

ファイル構成

新規 / 変更したファイル：

```
docs-checker/
├── .github/
│   └── workflows/
│       └── pr-review.yml          # 新規：Claude PR レビュー
├── lefthook.yml                  # 新規：pre-commit / pre-push 定義
├── backend/
│   ├── pyproject.toml            # ruff を dev 依存に追加、設定を追加
│   ├── api/, tests/              # ruff format / fix 適用済み
└── README.md                     # 開発フロー（インストール手順、Secret 設定）追記
```

Claude PR レビューの起動方法（initial setup）

リポジトリ単位で1回だけ：

1. **GitHub App インストール**：<https://github.com/apps/claude> で対象リポを選択
2. **OAuth トークン生成**：`claude setup-token` で `sk-ant-oat01-...` を取得
3. **Secret 登録**：Settings → Secrets → Actions → `CLAUDE_CODE_OAUTH_TOKEN` を追加

Plan / Pro / Max を契約していれば API キー不要、Plan の枠から消費される。

lefthook の起動方法

```
brew install lefthook
lefthook install
```

これで `.git/hooks/` に lefthook のラッパが置かれ、以降の commit / push でフックが走る。

緊急時のバイパス：

```
LEFTHOOK=0 git commit -m "..."  
git push --no-verify
```

検証

自動テスト

項目	結果
<code>uv run pytest</code>	14 件 pass (既存と同じ)
<code>uv run ruff check</code>	エラー無し
<code>uv run ruff format --check</code>	全 OK

手動確認

- ☐ `lefthook install` 後にダミーの commit でフックが走り lint チェックされること
- ☐ PR を出した時に `pr-review.yml` が起動すること (PR #2)
- ☐ @claude メンションでの追加レビュー (マージ後に確認)

確認観点 (architecture.md より)

- ☐ 環境変数をハードコードしてない (`CLAUDE_CODE_OAUTH_TOKEN` は GitHub Secrets 管理)
- ☐ エラーハンドリング: lefthook 失敗時は commit/push がブロックされる
- ☐ 自分で説明できる: lint 設定の各 select/ignore の理由が docstring または本文に記載

残課題・引き継ぎ

- frontend 側の lint (prettier / eslint) は phase 4 で SvelteKit セットアップと一緒に追加予定。lefthook には現状未登録。
- ruff の `select` は最小限のセット。phase 4 以降で `S` (bandit セキュリティ) や `RUF` の追加を検討。
- Claude PR レビューの Plan 枠消費は要観察。大きな PR が増えたら `--max-turns` を下げる調整が必要。
- 設計ドキュメント (phase-N.md) の PDF 生成も lefthook に組み込めるが、`pandoc + weasyprint` 依存が増えるので保留。